

Тестовое задание: Система управления мероприятиями (Event Manager)

Уровень: Middle/Senior Python Developer

Технологии: Django, DRF, PostgreSQL, Celery, Redis

Контекст

Вам необходимо разработать бэкенд для платформы, где организаторы создают мероприятия, а пользователи регистрируются на них. Система должна учитывать баланс бонусов, рейтинг организаторов и отправлять уведомления.

1. Модели данных (Models)

Спроектируйте модели в "models.py" со следующими полями:

Пользователь (Custom User)

- Расширить стандартную модель Django
- Добавить поле "bonus_balance" (Decimal, по умолчанию 0.00)
- Добавить поле "phone_number" (уникальный, необязательный)

Мероприятие (Event)

- Название, описание
- Дата и время начала/окончания
- Местоположение (адрес)
- Максимальное количество участников ("max_attendees", Integer, может быть NULL = безлимит)
- Стоимость участия ("price", Decimal, по умолчанию 0.00)
- Организатор (ForeignKey на User)
- Статус (Draft, Published, Canceled, Completed)
- Дата создания/обновления

Регистрация (Registration)

- Пользователь (ForeignKey)
- Мероприятие (ForeignKey)
- Дата регистрации
- Статус регистрации (Pending, Confirmed, Canceled, Attended)
- Использовано ли бонусов при оплате ("bonus_used" Boolean)

Отзыв (Review)

- Пользователь (ForeignKey)
- Мероприятие (ForeignKey)
- Рейтинг (Integer от 1 до 5)
- Текст отзыва
- Дата создания

2. Бизнес-логика (Core Logic)

Реализуйте класс (например, "EventService"), который содержит методы:

2.1. Регистрация на мероприятие "register_user_for_event(user, event_id)"

- Проверить, что мероприятие опубликовано и не завершено
- Проверить лимит мест (если есть)
- Проверить, не зарегистрирован ли пользователь уже
- Логика оплаты:
 - Если цена > 0:
 - Если "user.bonus_balance >= event.price", списать бонусы полностью (установить "bonus_used=True")
 - Иначе создать запись в модели "Registration" со статусом "Pending" (ожидает оплаты деньгами)
 - Если цена == 0, сразу статус "Confirmed" и начислить организатору +5 бонусов (как поощрение за бесплатное мероприятие)

2.2. Отмена регистрации "cancel_registration(user, registration_id)"

- Разрешена только за 2 дня до начала мероприятия (проверить "event.start_time")
- Если использовались бонусы — вернуть их пользователю
- Обновить статус "Registration" на "Canceled"

2.3. Получение списка мероприятий с рейтингом

- Вернуть список всех актуальных мероприятий (Published, дата окончания > сейчас)
- Добавить к каждому объекту:
 - "current_attendees" (количество подтвержденных регистраций)
 - "average_rating" (средний рейтинг из отзывов, если нет — 0)
 - "is_full" (булево, основанное на "max_attendees" и "current_attendees")
- Важно: Сделать это за 2 запроса к БД (без N+1), используя "annotate" и "prefetch_related"

3. API Endpoints (DRF)

Разработайте View-классы для:

3.1. "/api/events/"

- GET: Список мероприятий (используйте сериализатор, который отображает вычисленные поля из п.2.3)
- POST: Создание мероприятия (только для авторизованных пользователей, организатор проставляется автоматически)

3.2. "/api/events/{id}/register/"

- POST: Регистрация пользователя на мероприятие (вызов сервиса)
- Возвращать статус и информацию об оплате

3.3. "/api/profile/me/"

- GET: Профиль пользователя с его балансом и историей регистраций (последние 10)

4. Асинхронные задачи (Celery / DRF + Celery)

Настройте Celery.

Задача: Отправка напоминания за 1 день до начала мероприятия.

- Создайте периодическую задачу (или Celery Beat), которая раз в час проверяет мероприятия, начинающиеся через 24 часа
- Для каждого зарегистрированного пользователя (статус "Confirmed") отправляет уведомление. В реализации достаточно "print(f'Notify {user.email} about {event.title}')"
- Важно: Задача должна учитывать, что уведомление отправляется только 1 раз (добавьте поле "reminder_sent" в модель Event или в отдельную модель логов)

5. Тесты (Pytest)

Напишите тесты (хотя бы 3-4) на:

- Регистрацию пользователя при заполненных местах (должна выбросить исключение или вернуть ошибку)
- Проверку расчета "average_rating" через ORM
- Отмену регистрации (проверить возврат бонусов)
- Проверку, что организатор не может зарегистрироваться на свое же мероприятие